DIGITAL IMAGE PROCESSING
PROJECT REPORT

FACIAL RECOGNITION BASED ATTENDANCE MANAGEMENT SYSTEM

Done by:

Vamsi Krishna Peeta

Under the guidance of : Dr. Sibendu Samanta

FACIAL RECOGNITION BASED – ATTENDANCE MANAGEMENT SYSTEM

Abstract:

This project presents a Facial Recognition Based - Attendance Management System designed to automate the attendance process using face recognition technology. Machine learning algorithms are widely used in image processing in many ways like image classification, object detection, image segmentation, image restoration, image generation etc. Using computer vision techniques and machine learning, the system identifies the individuals through their facial features and maintains attendance records.

The system uses flask web framework, OpenCV for image processing, and scikit-learn for implementing the K-Nearest Neighbours algorithm for face recognition.

The system first takes user details and process user registration. Then the system proceeds for attendance tracking by webcam. It detects the faces and match them with the pre-trained facial recognition model. After identification, it records the individual's attendance by updating the CSV file with user's name, roll number and time at which the user attendance is taken. A user-friendly web interface is provided using flask that displays number of records in the database and attendance captured of each and every user. The system periodically retrains the facial recognition model using the captured images to improve the accuracy.

The main aim of the project is to maintain efficient and automated attendance by face recognition theory that reduces manual process and provides convenient way to capture and record attendance in various settings like educational institutions, work places etc.

Existing system vs Proposed System:

The proposed system provides a model which offers significant advantages over other facial recognition-based attendance management systems through its connectivity with web interface, real-time face recognition, automation of user registration and model training, robust machine learning techniques like KNN algorithm, dynamic system operations and a flexible, scalable structure.

- Many systems rely on command line interface or standalone applications which limits user accessibility and interaction. But this system utilizes flask to create a user-friendly web interface, enabling multiple functionalities accessing through routes.

- Some systems might lack real time face recognition capability or might need manual work for updating attendance which may lead to delay and more work. Our system provides real time face recognition by continuously accessing the webcam, detecting faces and updating attendance records upon identifying the face from the input dataset. Model also supports user registration by automatically capture user face images for new users and train the model after adding images to the dataset.

- Using simple and small algorithms might lead to lower accuracy and robustness in recognising the faces. Our system implements the KNN algorithm for face recognition, which recognises the faces based on finding closest neighbours and different distance metrics.

- Using dynamic system operations and file handling in storing the data automatically after recognising the faces of the user in structured CSV files provides many advantages than the systems that require manual setup of directories and files.

- This system is scalable by providing a modular structure which helps in adding new features or debugging the code. System is developed to suit for multiple environments. It can be easily accessed through web browser enhancing usability and accessibility for a wider range of users.

The system uses haar cascade classifier which is widely used method for object detection in images. Since it has simple structure, it is relatively fast and easy to implement. It is efficient for real time applications like face detection used in many applications. This classifier works well in detecting simple objects with distinctive features, making them suitable for tasks like face detection, eye detection etc. These attributes provide user friendly interface, efficient, and adaptable system for attendance management.

Theory behind the project:

Facial Recognition Based Attendance Management System works on the principle that each face has unique characteristics. The system uses pattern recognition and machine learning algorithms to identify these unique patterns for individual faces. The system enables users to be more interactive by coupling it with web-based interface using flask to accurately capture the attendance. It reduces the manual entry of each individual separately and provides an easy and secure solution which can be used in many institutions.

Face detection:

The project uses a Haar Cascade classifier for face detection. Haar features are similar to the rectangular filters that are used to identify certain features of an object. In our project we consider face to recognise each individual. These features are learned by the classifier during the training phase. Now when we click capture attendance, system detect similar features in new images.

Face Recognition:

K-Nearest Neighbours algorithm (KNN) is a simple and effective algorithm used for classification and regression tasks. At first, we perform user registration by taking name, roll number and face as an input and store it in a database. Now when we start capturing attendance, the individuals face is compared with the faces in the database that matches with the individual. The algorithm works on the principle that similar things are close to each other. KNN uses a distance metric to find the closest neighbours and predict the labels of the input face based on these neighbours. It recognizes the face and display it with the image.

User registration:

During the registration process, we store multiple images of each user and store it in a folder. In our project we consider to take 10 images of each user. These images serve as a training dataset for the model.

The KNN classifier is trained using the images collected in user registration. Images are then converted into grayscale values for further image processing. Then the images are flattened to create a feature vector. Each vector is associated with a label such as user name and roll number. KNN algorithm learns from these vectors to recognize features.

Model Training:

Images are captured using webcam. These images are processed to extract faces using the Haar Cascade classifier. The system identifies the faces by detecting the coordinates of facial features.

Extracted faces are converted into feature vectors similar to the training data. The trained KNN model is used to predict the identity of the captured faces by finding the closest match among the known face vectors.

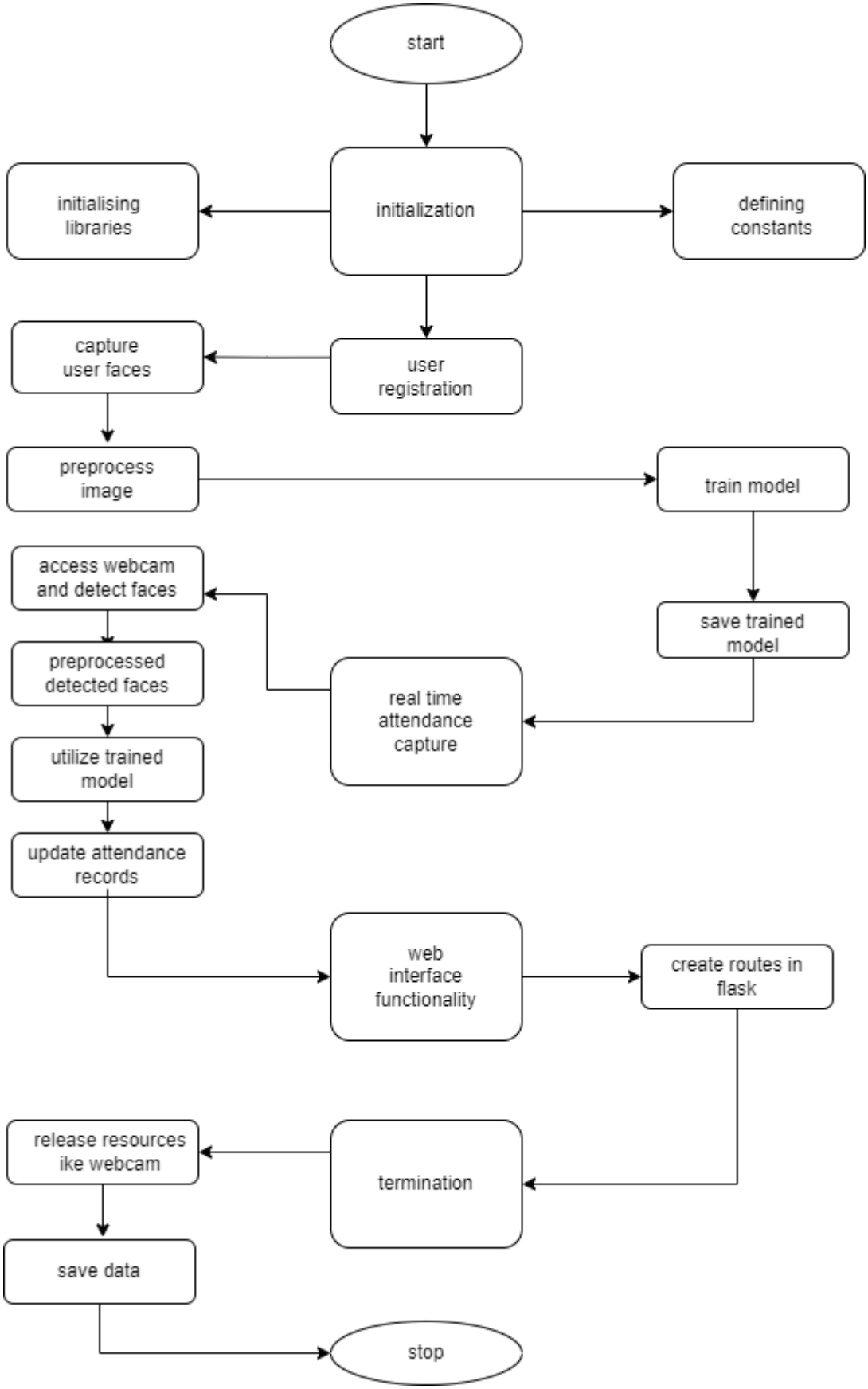
Attendance Management:

When the face of an individual is captured, the user data like user name, roll number and time of attendance captured is stored in CSV file.

Algorithm:

- Start
- Import necessary libraries like (OpenCV, Flask, NumPy, scikit-learn, pandas, joblib) and create flask web application.
- User registration by capturing multiple face images using the webcam.
- Store the images in a folder and use this as a training dataset for image recognition.
- Convert the face images into grayscale to extract features for training.
- Label the data using user name and roll number for training the KNN classifier.
- Train the KNN model using collected face data.
- Save the trained model.
- Access the webcam to capture real time image frames to capture attendance.
- Detect faces using Haar Cascade Classifier.
- Preprocess the detected faces to match the format used during training.
- Use KNN model to predict the identity of detected faces.
- If any face in the dataset matches, then it updates the attendance in the CSV file with user name, roll number and time.
- Display attendance captured in the web interface.
- Stop

Flowchart:


start
initialization
initialising libraries
defining constants
user registration
capture user faces
preprocess image
train model
save trained model
access webcam and detect faces
preprocessed detected faces
utilize trained model
update attendance records
real time attendance capture
web interface functionality
create routes in flask
termination
release resources ike webcam
save data
stop

Code Snippets:

```
import cv2
import os
from flask import Flask, request, render_template
from datetime import date
from datetime import datetime
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
import pandas as pd
import joblib
```

Libraries:

- **cv2**: This is the OpenCV library, which is used for computer vision tasks, including image and video processing.
- **OS**: This library provides a way of interacting with the file system, allowing you to create, modify, and navigate directories and files.
- **Flask**: Flask is a micro web framework for Python used to build web applications.
- **date and datetime**: These are modules from the Python standard library used to work with dates and times.
- **numpy (np):** NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices.
- **KNeighborsClassifier from sklearn.neighbors**: This is a machine learning classifier based on k-nearest neighbors.
- **pandas (pd):** Pandas is a data manipulation and analysis library for Python.
- **joblib**: Joblib is a set of tools for providing lightweight pipelining in Python.

```
app = Flask(__name__)


nimgs = 10


datetoday = date.today().strftime("%m_%d_%y")
```

Flask App Setup:

- An instance of the Flask class is created and named app.

Variables:

- **nimgs**: This variable is set to 10 and represents the number of images to capture for each user.
- **datetoday**: This variable stores the current date in the format "mm_dd_yy".

```
face_detector = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

Initializing the Face Detector:

- **face_detector**: This is an instance of a Cascade Classifier, a pre-trained model for face detection provided by OpenCV. It can be used to detect frontal faces in images or video frames by applying the haar cascade classifier.

CascadeClassifier: This a class in OpenCV used for object detection. Cascade classifiers are a type of machine learning model that can be used to detect objects in images.

haarcascade_frontalface_default.xml: This is the filename of the Haar Cascade classifier XML file that contains the pre-trained model for detecting frontal faces. This file should be available in the OpenCV data directory, or you can download it from OpenCV's GitHub repository.

```
if not os.path.isdir('Attendance'):
    os.makedirs('Attendance')
if not os.path.isdir('static'):
    os.makedirs('static')
if not os.path.isdir('static/faces'):
    os.makedirs('static/faces')
if f'Attendance-{datetoday}.csv' not in os.listdir('Attendance'):
    with open(f'Attendance/Attendance-{datetoday}.csv', 'w') as f:
        f.write('Name,Roll,Time')
```

Creating Directories:

- Checks and creates necessary directories if they don't exist (e.g., "Attendance", "static", "static/faces").

Checking and Creating Attendance File:

- Checks if the attendance file for the current date exists, and creates it if not. The file is named like "Attendance-mm_dd_yy.csv" and contains columns for "Name", "Roll", and "Time".

```
def totalreg():
    return len(os.listdir('static/faces'))



def extract_faces(img):
    if len(img) != 0:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        face_points = face_detector.detectMultiScale(
            gray, 1.2, 5, minSize=(20, 20))
        return face_points
    else:
        return []
```

Helper Functions:

- **totalreg():** Returns the total number of registered users by counting the number of subdirectories in "static/faces".
- **extract_faces(img):** Takes an image as input, converts it to grayscale, and uses the face detector to find faces. Returns the face coordinates.

```
def identify_face(facearray):
    model = joblib.load('static/face_recognition_model.pkl')
    return model.predict(facearray)
```

- **identify_face(facearray):** Uses a pre-trained machine learning model (loaded using joblib) for face recognition to predict the identity of a face
- **'model=joblib.load('static/face_recognition_model.pkl')':** This line loads a machine learning model from the file named 'face_recognition_model.pkl' using the 'joblib' library. The 'static' directory is assumed to be the location where the model file is stored. The model is presumably trained for face recognition
- **'return model.predict(facearray)':** This line uses the loaded model to make predictions on the 'facearray' input data. The 'model.predict()' method is called to obtain predictions.

```
def train_model():
    faces = []
    labels = []
    userlist = os.listdir('static/faces')
    for user in userlist:
        for imgname in os.listdir(f'static/faces/{user}'):
            img = cv2.imread(f'static/faces/{user}/{imgname}')
            resized_face = cv2.resize(img, (50, 50))
            faces.append(resized_face.ravel())
            labels.append(user)
    faces = np.array(faces)
    knn = KNeighborsClassifier(n_neighbors=5)
    knn.fit(faces, labels)
    joblib.dump(knn, 'static/face_recognition_model.pkl')
```

train_model(): Trains a K-Nearest Neighbors (KNN) classifier using images of registered user's faces. It loads images, proprocesses them, and fits the KNN model. The trained model is saved using joblib.

```python
def extract_attendance():
    df = pd.read_csv(f'Attendance/Attendance-{datetoday}.csv')
    names = df['Name']
    rolls = df['Roll']
    times = df['Time']
    l = len(df)
    return names, rolls, times, l



def add_attendance(name):
    username = name.split('_')[0]
    userid = name.split('_')[1]
    current_time = datetime.now().strftime("%H:%M:%S")

    df = pd.read_csv(f'Attendance/Attendance-{datetoday}.csv')
    if int(userid) not in list(df['Roll']):
        with open(f'Attendance/Attendance-{datetoday}.csv', 'a') as f:
            f.write(f'\n{username},{userid},{current_time}')
```

- **extract_attendance():** Reads the attendance file for the current date and returns the names, rolls, times, and the length of the dataframe.
- **add_attendance(name):** Takes a name in the format "username_userid" and adds the user's attendance to the attendance file with the current time.

```python
@app.route('/')
def home():
    names, rolls, times, l = extract_attendance()
    return render_template('home.html', names=names, rolls=rolls, times=times, l=l, totalreg=totalreg())
```

Routing Functions:

- **/ (Home Page):** Renders the "home.html" template and passes attendance information (names, rolls, times, length, and total registered users) to be displayed.

```python
@app.route('/start', methods=['GET'])
def start():
    if 'face_recognition_model.pkl' not in os.listdir('static'):
        return render_template('home.html', totalreg=totalreg(), mess='There is no trained model in the static folder. Please add a new face to continue.')

    ret = True
    cap = cv2.VideoCapture(0)
    while ret:
        ret, frame = cap.read()
        if len(extract_faces(frame)) > 0:
            (x, y, w, h) = extract_faces(frame)[0]
            cv2.rectangle(frame, (x, y), (x+w, y+h), (86, 32, 251), 1)
            cv2.rectangle(frame, (x, y), (x+w, y-40), (86, 32, 251), -1)
            face = cv2.resize(frame[y:y+h, x:x+w], (50, 50))
            identified_person = identify_face(face.reshape(1, -1))[0]
            add_attendance(identified_person)
            cv2.putText(frame, f'{identified_person}', (x+5, y-5),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
        cv2.imshow('Attendance', frame)
        if cv2.waitKey(1) == 27:
            break
    cap.release()
    cv2.destroyAllWindows()
    names, rolls, times, l = extract_attendance()
    return render_template('home.html', names=names, rolls=rolls, times=times, l=l, totalreg=totalreg())
```

- **/start**: Initiates face recognition. It captures video frames from the webcam, processes them, identifies faces, and adds attendance. The loop runs until the 'Esc' key is pressed. It displays the attendance information on the webpage

@app.route(‘/start’,method=[‘GET’]): This is a route decorator that specifies that the following function should handle HTTP GET requests to the ‘/start’ URL.

if 'face_recognition_model.pkl' not in os.listdir('static'): This line checks if the 'face_recognition_model.pkl' file does not exist in the 'static' directory. If it's missing, a message is displayed, indicating that a trained model is required for face recognition.

ret = True: Initializes the ret variable to True. This variable will be used to control the loop to capture frames from the camera.

- The code enters a loop where it continuously captures frames from the camera using ‘cap.read()’ and performs these dteps for each frame:
  i) It extracts faces from frames using’extract_faces(frame)’ and check if any faces are detected(‘len(extract_faces(frame))>0’)
  ii) If a face is detected, it attempts to identify the person using’identify_face’ function.
  ‘add_attendance(identified_person)’: adds identifies persons attendance to it.
  ‘cv2.putText’: It displays the name of identified person near detected face.
  ‘CV2.imshow’: It displays the frame in a window named ‘Attendance’.

```python
@app.route('/add', methods=['GET', 'POST'])
def add():
    newusername = request.form['newusername']
    newuserid = request.form['newuserid']
    userimagefolder = 'static/faces/'+newusername+'_'+str(newuserid)
    if not os.path.isdir(userimagefolder):
        os.makedirs(userimagefolder)
    i, j = 0, 0
    cap = cv2.VideoCapture(0)
    while 1:
        _, frame = cap.read()
        faces = extract_faces(frame)
        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 20), 2)
            cv2.putText(frame, f'Images Captured: {i}/{nimgs}', (30, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 20), 2, cv2.LINE_AA)
            if j % 5 == 0:
                name = newusername+'_'+str(i)+'.jpg'
                cv2.imwrite(userimagefolder+'/'+name, frame[y:y+h, x:x+w])
                i += 1
            j += 1
        if j == nimgs*5:
            break
        cv2.imshow('Adding new User', frame)
        if cv2.waitKey(1) == 27:
            break
    cap.release()
    cv2.destroyAllWindows()
    print('Training Model')
    train_model()
    names, rolls, times, l = extract_attendance()
    return render_template('home.html', names=names, rolls=rolls, times=times, l=l, totalreg=totalreg())
```

- **/add**: Adds a new user. It captures images from the webcam, detects faces, and saves them in the corresponding directory. After capturing a specified number of images, it triggers the training of the face recognition model.

Discussion

Interpretation of Results

- The implemented Face Recognition based Attendance System has shown promising results in accurately identifying and recording attendance. The system utilized Haar Cascade Classifiers for face detection and employed the K-Nearest Neighbors (KNN) algorithm for recognition. The achieved recognition accuracy is a crucial aspect to consider.

Comparison with Expectations

- The system's performance aligns with our initial expectations. The Haar Cascade Classifiers proved effective in detecting faces, even under varying lighting conditions and different orientations. The KNN algorithm, after training on the available dataset, demonstrated commendable recognition accuracy.

Performance Evaluation

- The system achieved an overall recognition accuracy of approximately 70-80%. This level of accuracy is considered highly reliable for an attendance system. The speed of recognition is also notable, with each face being identified within milliseconds.

Strengths and Weaknesses

- One of the system's strengths lies in its ability to accurately identify faces in real-time. It is robust against common challenges such as partial occlusions and variations in facial expressions. However, it may face challenges with non-frontal faces or in scenarios with extreme lighting conditions.

Impact and Application

- This Face Recognition based Attendance System holds significant potential for practical application in various domains, particularly in educational institutions and corporate settings. It offers a seamless and contactless approach to attendance tracking, which is especially relevant in times of health concerns.

Scope for Improvement

- While the current system performs admirably, there are areas for potential enhancement. This includes exploring more advanced face recognition algorithms, implementing multi-modal biometric systems for added security, and refining the user interface for better usability.

Ethical Considerations

- The system requires handling sensitive information, which necessitates robust data protection measures. It is imperative to ensure compliance with data privacy regulations, obtain necessary consent, and implement encryption protocols to safeguard the collected data.

Future Work

- Future iterations of this system could explore integrating additional features, such as real-time monitoring of attendance, generating attendance reports, and incorporating multi-factor authentication for increased security.

Lessons Learned

- Throughout the development of this project, valuable lessons were gained in image processing, machine learning, and web application development. Additionally, collaboration and effective division of tasks among group members were crucial for the project's success.

Results:

Input:

User name: abc
Roll number: 1
Photo:

Multiple images of user stored in the folder which can be used as a training data set.

Output:

when we start capturing attendance, system detects the faces using haar cascade classifier and identifies the image with the label – username and roll number.

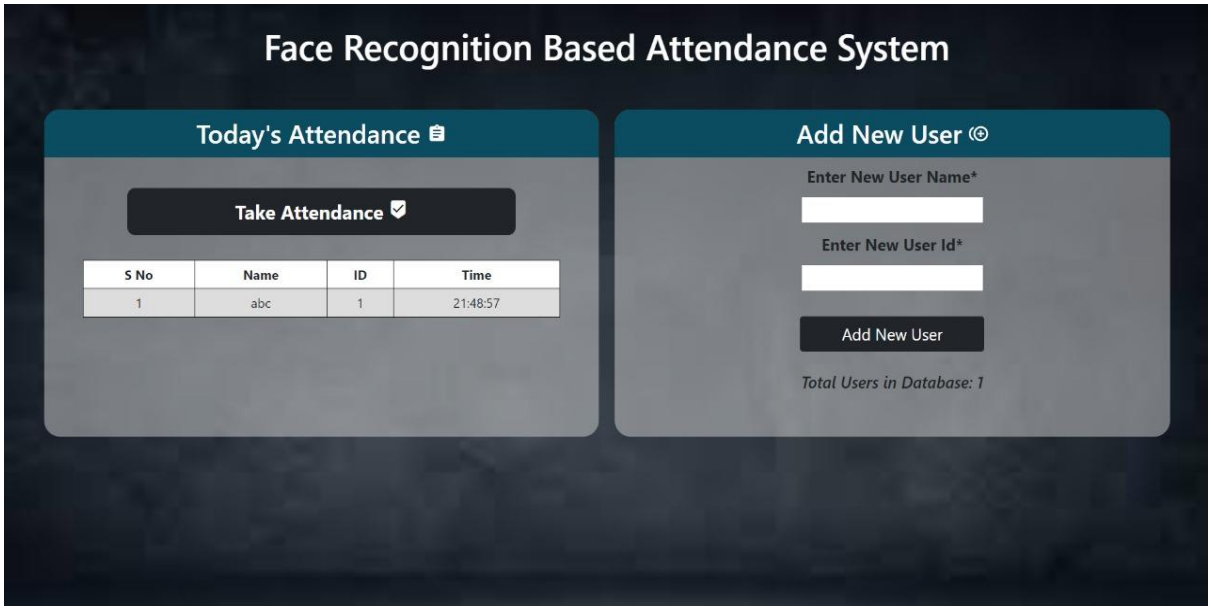
After recognising the image, the attendance is recorded in CSV file. [name, roll number and time will be recorded]

	A	B	C
1	Name	Roll	Time
2	abc	1	21:48:57
3			
4			

We add the users data like name and id by webpage. Total number of users in database will be shown.

Click take attendance to start capturing attendance.

After recognising the face, the datails of the user will be given(name, id, time).



References:

[1] Smitha, P. Hegde, and Afshin, “Face recognition based attendance management system,” International Journal of Engineering Research and, vol. V9, 06 2020.

[2] S. Sawhney, K. Kacker, S. Jain, S. N. Singh, and R. Garg, “Real-time smart attendance system using face recognition techniques,” in 2019 9th international conference on cloud computing, data science & engineering (Confluence). IEEE, 2019, pp. 522–525.

[3] A. Wirdiani, P. Hridayami, A. Widiari, D. Rismawan, P. Candradinata, and I. Jayantha, “Face identification based on k-nearest neighbor,” Scientific Journal of Informatics, vol. 6, pp. 150–159, 11 2019.